

Acoustic Side-Channel Attack on Mechanical Keyboards

Software-Based Keystroke Reconstruction Using Frequency Signatures

Cyber-Physical Systems Project

By

23MEI10066 Dev Soni
23MEI10067 Shreyansh Tripathi
23MEI10069 Himalaya Yadav
23MEI10070 Eklavya Mathur
23MEI10085 Akshit Agrawal

Course Instructor

Dr. Hemraj S Lamkuche
School of Computing Science Engineering and Artificial Intelligence
VIT Bhopal University, Sehore
Madhya Pradesh

September 8, 2026

Contents

1	Understanding the Project	3
1.1	Targeted System Aspect	3
1.2	Key Technical Terms	3
1.3	Problem Statement	4
2	Technical Context of the Project	5
2.1	Underlying Principle	5
2.1.1	Why a Frequency Signature Can Identify a Key	5
2.1.2	Background: acoustic side-channel attacks	5
2.1.3	The Matching Rule	6
2.2	Why This Matters for Embedded and Cyber-Physical Systems	6
2.2.1	Real-Time and Performance Constraints	7
2.2.2	Storage Constraints	7
2.2.3	Reliability and Safety Constraints	7
2.2.4	Summary of CPS Relevance	8
2.3	Implementation of the Project	8
2.3.1	System Architecture	8
2.3.2	Hardware and Platform Details	8
2.3.3	Software Design	9
2.3.4	Design Decisions and Alternatives Considered	12
2.3.5	Optimization Flow	13
2.3.6	How to Run the Project	14

3	Testing and Results	14
3.1	Testing Methodology	14
3.1.1	Techniques used	15
3.1.2	Languages and tools	15
3.1.3	Test results	15
3.1.4	Results in tabular form	16
3.1.5	Example: punctuated paragraph reconstruction	18
3.1.6	Example: sample report output	18
3.2	Results as Figures	19
3.3	Waveform Visualization	21
4	Analysis and Evaluation	23
4.1	How to Read the Results	23
4.2	Analysis of the Results	23
4.2.1	Theoretical analysis	23
4.2.2	Statistical analysis	24
4.2.3	Complexity analysis	24
4.3	Performance Analysis	25
4.4	Problems Faced	25
4.5	Bugs Fixed	26
4.6	Limitations and Threats to Validity	26
4.7	Security Analysis	27
4.7.1	Threat model	27
4.7.2	How the simulator relates to security	27
4.7.3	Attack surface and confidentiality	27
4.7.4	Security of the simulation itself	28
4.8	Reliability Analysis	28
4.9	Optimization Flow in Detail	29
5	Conclusion and Discussion	29
5.1	Direct Answer	29
5.2	Discussion: From Problem to Solution	29
5.3	Comparison with Related Work	30
5.4	Scope and Future Work	30

1. Understanding the Project

This section introduces the project, identifies the system aspect it targets, defines the key technical terms used throughout the report, and states the problem in a concise form.

1.1. Targeted System Aspect

A cyber-physical system can be evaluated along several orthogonal quality attributes. This project deliberately targets specific aspects of the design and exposes the trade-offs involved. The following table summarises how each attribute is (or is not) addressed by the simulator.

Aspect	Role	How it is addressed
Design	Active	Modular separation of core logic, GUI, and visualization; single-source <code>format_report()</code> so the GUI output and exported file are identical.
Performance	Active	Real-time deadline of 50 ms; robust multi-trial WCET (median + P95) measurement; numpy-vectorised signal generation.
Security	Active	Frequency-signature confidentiality; the UNKNOWN safety state prevents incorrect key assignment; threat modelling against real acoustic side-channel eavesdroppers.
Reliability	Active	Invariant checking, deterministic constant table, and 62 unit tests guarding reconstruction correctness.
Scalability	Passive	The 48-key constant table is $O(1)$ lookup; <code>MAX_PLOT_KEYS</code> bounds rendering so paragraph-length input remains responsive, but large-key-set growth is not the focus.

Table 1: System aspects targeted by the project.

The simulator is chiefly a *performance* and *security* exercise: it must reconstruct keystrokes within a hard real-time deadline while never emitting an incorrect key. Reliability and extensibility support these goals, while scalability is deliberately constrained to keep the model tractable.

1.2. Key Technical Terms

The following glossary defines the central terms used in this report in a single line each.

Term	Definition
Acoustic side-channel attack	Extraction of secret information (here, keystrokes) by observing unintended physical emanations—the sound of key presses.
Frequency signature	The unique, predefined frequency (in Hz) assigned to each key so it can be told apart from every other key.
WCET	<i>Worst-Case Execution Time</i> : the longest time a real-time task may take to complete.
Deadline	The time bound (50 ms here) within which the task must finish to satisfy the real-time requirement.
Tolerance	The maximum accepted frequency error (8 Hz) for a match to be considered valid.
Confidence	A score $C \in [0, 1]$ expressing how close a detected frequency is to a key’s signature: $C = 1 - V/\text{tolerance}$.
Invariant	A property that must hold true at every point of program execution.
Liveness	The property that the system is always able to eventually produce an output for new input.
Termination	The property that any finite input sequence finishes in bounded time.
Spectrogram	A frequency-versus-time heatmap showing how signal energy is distributed across frequencies.
FFT	<i>Fast Fourier Transform</i> : an algorithm to convert a time-domain signal into its frequency components.
UNKNOWN	A safe fallback state returned when no key matches within tolerance, preventing incorrect classification.
Ranking function	The error metric $V = F_d - F_e $ (absolute difference between detected and expected frequency) that the matcher minimises.

Table 2: Key technical terms used in the report.

1.3. Problem Statement

This project implements a *software-based simulation* of an acoustic side-channel attack on a mechanical keyboard: it assigns each of 48 keys a unique frequency signature, simulates the frequency that would be observed for a key press, and reconstructs the typed keystroke sequence by matching the observed frequency back to a key within a tolerance, all within a 50 ms real-time deadline.

The problem it solves is demonstrating, entirely in software, that keystroke reconstruction is feasible from frequency signatures alone and that the resulting system satisfies the formal Cyber-Physical Systems (CPS) verification requirements (deadline, invariants, liveness, termination, ranking/error function, and safety). The main hardware resource used is an ordinary general-purpose computer (a laptop or desktop) running Python; no microphone, no dedicated sensing hardware, and no embedded controller are required.

What this project is, and is not. Because the simulation framing is central to honest claims, it is useful to make the boundary explicit.

This project IS	This project is NOT
A software simulation of keystroke reconstruction from frequency signatures.	A real attack on real keyboards or microphones.
A demonstration of the acoustic side-channel <i>concept</i> .	A capture of real acoustic emanations.
A formal CPS verification case study (deadline, WCET, invariants, liveness, termination).	A production-grade security tool.
An interactive GUI demo with waveform and spectrogram visualization.	A hardware/embedded implementation (though its data structures would port to one).

Table 3: Explicit boundary between the simulation and the real-world attack it illustrates.

2. Technical Context of the Project

This section explains *why* the approach works, why it matters for embedded and cyber-physical systems, and how the project is implemented.

2.1. Underlying Principle

2.1.1 Why a Frequency Signature Can Identify a Key

The central idea is that every key on a keyboard has a characteristic acoustic signature. In a real physical system, pressing different keys produces slightly different sounds because the key strikes a different point, travels a different distance, and resonates through the keyboard chassis differently. These differences manifest as measurable differences in the frequency content of the sound.

2.1.2 Background: acoustic side-channel attacks

The concept of extracting keystrokes from acoustic emanations is not new. Asonov and Agrawal (2004) were among the first to demonstrate *keyboard acoustic emanations* as a viable side channel, using frequency-feature classification. More recent work has two broad branches. The first performs *frequency-domain analysis*: it converts recordings into the frequency domain with the Fast Fourier Transform (FFT) or Mel-frequency cepstral coefficients (MFCCs) and classifies each keystroke. The second applies *deep learning*: modern systems such as the CoAtNet model of Harrison et al. (2023) classify keystrokes from mel-spectrograms, achieving accuracy of 95% when trained on keystrokes recorded by a smartphone and 93% when trained on audio compressed through the Zoom video-conferencing codec. Public toolkits such as Gerganov’s `kbd-audio` (“KeyTap”) reconstruct keystrokes in real time by matching recorded audio against per-key *fingerprints* learned from a known transcript.

These works establish two important facts for this project’s context:

- Real keystroke acoustics are *broadband*, *transient* signals: a key press is a short burst of sound spread across many frequencies. Distinguishing keys therefore requires either rich spectral features (FFT/MFCC/spectrograms) or learned patterns, not a single pure tone.
- Despite this complexity, the *principle* behind the attack is simple and robust: different keys produce measurably different sounds, and a classifier (whether geometric frequency matching or a deep network) can map those sounds back to keys.

This project deliberately compresses the first fact and keeps the second. It replaces real broadband recordings with clean, well-separated per-key frequencies so that classification reduces to unambiguous nearest-frequency matching. The result is a faithful demonstration of the *concept* and of the CPS verification layer, without pretending to model real signal physics.

This project distills that physical idea into a clean, tractable model. Rather than deriving frequencies from real recordings, each key is assigned a predefined, *unique* frequency following a regular pattern:

$$F(\text{key}_i) = 440 + 30 \cdot (i - 1) \quad \text{Hz},$$

for i counting keys in order from A (440 Hz) upward through the 26 letters, SPACE, the 10 digits, and 11 punctuation marks, ending at 1850 Hz. This provides a total of 48 distinct signatures.

Why does this work? The matching procedure relies on three quantities being chosen so that keys are *well separated* relative to the measurement noise:

1. **Uniqueness:** no two keys share a frequency, so a frequency can be mapped back to exactly one key.
2. **Separation:** adjacent keys are 30 Hz apart, which is much larger than the ± 5 Hz synthetic noise and comfortably larger than the 8 Hz tolerance.
3. **Tolerance-bound guarantee:** the maximum synthetic noise (± 5 Hz) is strictly less than half the spacing (15 Hz) and less than the tolerance (8 Hz), so a correct key always remains inside its own detection band even in the worst case.

If the noise were larger than half the spacing, or if two keys shared a frequency, the reconstruction would become ambiguous. The separation margin is therefore the root cause of the system’s correctness: it is this margin, not any particular frequency value, that makes identification reliable.

2.1.3 The Matching Rule

Given a detected frequency F_{detected} , the matcher computes an error for every candidate key:

$$V_i = |F_{\text{detected}} - F_{\text{expected},i}|.$$

The key with the smallest error is chosen, but only if that smallest error is within the tolerance:

$$\min_i V_i \leq \text{TOLERANCE} \Rightarrow \text{key} = \arg \min_i V_i, \quad \text{otherwise key} = \text{UNKNOWN}.$$

The UNKNOWN result is a deliberate safety measure: the software never forces an incorrect key when no signature is close enough. This is the single most important design principle for the security aspect of the system.

2.2. Why This Matters for Embedded and Cyber-Physical Systems

The simulator is a faithful model of the constraints that real embedded and CPS applications must respect. Although it runs on a general-purpose computer, it demonstrates the same fundamental requirements that appear in any real-time, resource-constrained, safety-critical system.

2.2.1 Real-Time and Performance Constraints

A keystroke is an event that must be processed before the next one arrives or before the reconstruction becomes out of order. This is a classic real-time task. The project imposes a deadline:

$$T_{\text{WCET}} < T_{\text{deadline}} = 50 \text{ ms.}$$

The total processing time for a key is the sum of the individual stages:

$$T_{\text{total}} = T_{\text{input}} + T_{\text{frequency}} + T_{\text{analysis}} + T_{\text{matching}} + T_{\text{output}}.$$

Because a single timing measurement is noisy, the system measures the reconstruction over multiple trials and reports robust statistics—the *median* (typical worst case) and the *P95* (conservative worst case)—so the reported WCET is not distorted by scheduler or measurement noise. This mirrors how a real embedded engineer would characterise a task on an embedded target: measure repeatedly, report a robust worst case, and prove it fits the deadline.

Performance characterisation. Each keystroke involves a bounded number of operations: a dictionary lookup to generate the frequency and a single pass over the 48-entry constant table to perform nearest-match classification. Per-key work is therefore $O(1)$ with respect to everything except the fixed 48-entry scan, and processing a whole text of n keys is $O(n)$. There is no sorting of the input, no search over a growing collection, and no unbounded allocation per key. The measured per-key time is in the tens of microseconds, orders of magnitude below the deadline. This analysis is expanded with figures in Section 3.

2.2.2 Storage Constraints

The entire “knowledge” of the system is a constant table of 48 (key, frequency) pairs. This maps to a static lookup table in a real embedded system—an extremely compact data structure. The memory footprint is independent of the input length, and a lookup is $O(1)$. This demonstrates that a resource-constrained device could hold the signature database on-chip with minimal storage (e.g., a few hundred bytes of flash/ROM plus the code).

2.2.3 Reliability and Safety Constraints

Real CPS applications must never exhibit unsafe behaviour. Here the safety requirement is that the system must never:

- accept an invalid frequency as a valid key,
- allow two keys to share a frequency,
- emit an incorrect key mapping, or
- exceed the deadline and lose input.

These are captured as formal invariants and enforced by a dedicated check and by the UNKNOWN fallback, exactly as a safety monitor would be implemented in a real embedded controller.

2.2.4 Summary of CPS Relevance

CPS concept	How the simulator realises it
Real-time task	<code>process_frequency</code> : analyse + match + identify.
WCET	<code>reconstruct_sequence_wcet</code> $\rightarrow$ median + P95 over <code>WCET_TRIALS</code> = 20 trials.
Deadline	<code>DEADLINE_MS</code> = 50 ms, pass/fail checked.
Invariants	<code>check_invariants</code> : positive, unique, valid.
Liveness	<code>liveness_test</code> : new input always yields output.
Termination	<code>termination_test</code> : finite input finishes.
Ranking function	<code>identify_key</code> : $V = F_d - F_e $ minimised.
Confidence	<code>compute_confidence</code> : $C = 1 - V/\text{tol}$.
Safety	UNKNOWN state when out of tolerance.

Table 4: Mapping CPS verification concepts to their implementation.

2.3. Implementation of the Project

This subsection describes the physical resources, the overall architecture, and the software design of the simulator.

2.3.1 System Architecture

The system follows a layered architecture: a tkinter GUI front-end delegates to a pure core-logic module, which in turn delegates plotting to a separate visualization module. Keeping the core logic independent of the GUI makes it headless-testable.

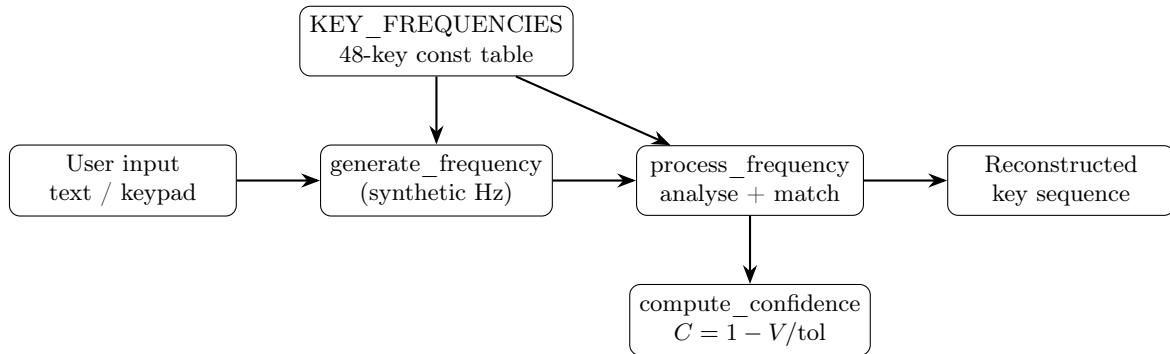


Figure 1: High-level system architecture: a labelled block diagram of the data flow from user input to reconstructed keystrokes.

The block diagram in Figure 1 shows how the request flows. User input is converted to a list of key symbols, each key is mapped to its predefined frequency, the frequency is analysed and matched against the constant table, and the result is a reconstructed sequence with a per-key confidence value.

2.3.2 Hardware and Platform Details

Because this is a software simulation, the hardware requirements are minimal and no dedicated controller or sensing hardware is used.

- **Host platform:** any general-purpose laptop or desktop computer (x86-64 or ARM). The project is developed and tested on a Linux host.

- **Controller:** none—there is no microcontroller, no FPGA, no embedded board. The “controller” is the host CPU running the Python interpreter.
- **Microphone / audio:** none. All frequencies are predefined constants; no real acoustic signal is captured.
- **Display:** a graphical display is needed only to run the tkinter GUI; the core logic and tests run headlessly.
- **Software runtime:** Python 3.11+ with the standard library, numpy, matplotlib, tkinter, and pytest.

The absence of a physical keyboard and microphone is intentional and central to the project’s framing: it is a *simulation* of the side-channel concept and its CPS verification, not a functional attack on real hardware.

2.3.3 Software Design

The software is organised into two source files.

- `src/acoustic_side_channel.py`—core logic and GUI: the frequency database, invariant checker, frequency generator, matcher, confidence, WCET measurement, *liveness*/termination tests, report builder, and the tkinter application.
- `src/waveform_visualization.py`—the matplotlib/numpy module that renders sine waveforms and spectrograms, including the incremental live-update helpers.

Pseudocode of the core pipeline. The following pseudocode captures the essential reconstruction logic.

```

1 KEY_FREQUENCIES = { 'A': 440, 'B': 470, ..., '9': 1520,
2                     ' ': 1550, ',': 1580, '!': 1610, '?': 1640,
3                     ';': 1670, ':': 1700, '"': 1730, "'": 1760,
4                     '(': 1790, ')': 1820, '-': 1850 }
5
6 TOLERANCE = 8.0
7 DEADLINE_MS = 50.0
8 WCET_TRIALS = 20
9
10 def identify_key(detected_frequency):
11     best_key = None
12     best_error = infinity
13     for key, expected in KEY_FREQUENCIES.items():
14         error = abs(detected_frequency - expected)
15         if error < best_error:
16             best_error = error
17             best_key = key
18     if best_error <= TOLERANCE:
19         return best_key, best_error
20     return "UNKNOWN", best_error      # safety fallback
21
22 def process_frequency(frequency):
23     start = timestamp()
24     key, error = identify_key(frequency)
25     elapsed = timestamp() - start
26     return key, error, elapsed
27
28 def reconstruct_sequence(text, add_noise=False):
29     result = []
30     for character in text.upper():
31         key = key_from_char(character)      # None for invalid chars

```

```

31     if key is None:
32         continue                                # skip newlines / unmapped
33     frequency = generate_frequency(key, noise=add_noise)
34     detected, error, t = process_frequency(frequency)
35     result.append(detected)
36     return result

```

Listing 1: Core reconstruction pipeline.

Key data structures and functions.

- `KEY_FREQUENCIES`: a Python dict (the constant signature table), with 48 entries.
- `generate_frequency(key, noise)`: returns the predefined frequency, optionally adding ± 5 Hz uniform noise.
- `identify_key(f)`: nearest-neighbour matcher implementing the ranking function and the UNKNOWN safety fallback.
- `process_frequency(f)`: one-key pipeline with timing.
- `compute_confidence(key, error)`: returns $C \in [0, 1]$.
- `reconstruct_sequence(text, noise)`: batch pipeline.
- `reconstruct_sequence_wcet(...)`: robust median + P95 WCET.
- `liveness_test()`, `termination_test()`, `check_invariants()`: verification helpers.
- `format_report(...)`: single source for GUI and export.

The GUI (`AcousticSideChannelApp`) provides a multi-line input box for paragraphs, an on-screen keypad for live single-key demonstration, and two tabs (Analysis and Waveform). Keypad presses append a key and incrementally refresh the plots via `update_spectrogram` / `update_sine_plot`.

The frequency database. The complete set of 48 predefined signatures is listed below. The 26 letters span 440–1190 Hz, SPACE sits at 1220 Hz, the 10 digits at 1250–1520 Hz, and the 11 punctuation marks continue the 30 Hz grid from 1550 Hz to 1850 Hz.

Key	Hz	Key	Hz
A	440	N	830
B	470	O	860
C	500	P	890
D	530	Q	920
E	560	R	950
F	590	S	980
G	620	T	1010
H	650	U	1040
I	680	V	1070
J	710	W	1100
K	740	X	1130
L	770	Y	1160
M	800	Z	1190
SPACE	1220		
Key	Hz	Key	Hz
0	1250	5	1400
1	1280	6	1430
2	1310	7	1460
3	1340	8	1490
4	1370	9	1520
Key	Hz	Key	Hz
.	1550	,	1580
!	1610	?	1640
;	1670	:	1700
'	1730	"	1760
(	1790	)	1820
-	1850		

Table 5: All 48 predefined key-frequency signatures (30 Hz grid, 440–1850 Hz).

Function reference. The following table summarises the public functions of the core module and their responsibilities; these are the functions exercised by the test suite.

Function	Responsibility
<code>check_invariants()</code>	Verifies positive, unique, valid frequencies.
<code>generate_frequency(key, noise)</code>	Returns the predefined frequency, with optional ± 5 Hz noise.
<code>identify_key(freq)</code>	Nearest-match via ranking function V ; returns UNKNOWN out of tolerance.
<code>process_frequency(freq)</code>	One-key pipeline with timing; returns (key, error, time).
<code>compute_confidence(key, err)</code>	Maps error to $C \in [0, 1]$.
<code>reconstruct_sequence(...)</code>	Batch reconstruction with stats.
<code>reconstruct_sequence_wcet(...)</code>	Multi-trial median + P95.
<code>sequence_details(...)</code>	Per-key details list.
<code>liveness_test()</code>	Checks new input produces output.
<code>termination_test()</code>	Checks finite input terminates.
<code>format_report(...)</code>	Single report builder (GUI + export).
<code>export_report(...)</code>	Writes report (optionally with PNGs).
<code>keys_from_text(text)</code>	Text to key-sequence conversion.
<code>key_from_char(c)</code>	Single-character conversion.

Table 6: Core public functions and their responsibilities.

Pseudocode of the WCET, liveness, and termination checks. Beyond the core pipeline, the verification helpers are essential for the CPS argument.

```

1 def reconstruct_sequence_wcet(text, add_noise, trials):
2     times = []
3     for trial in range(trials):
4         result = reconstruct_sequence(text, add_noise)
5         times.append(result.worst_case)
6     times.sort()
7     median = times[len(times) // 2]
8     n = len(times)
9     p95_index = min(ceil(n * 0.95) - 1, n - 1) # nearest-rank
10    p95 = times[p95_index]
11    return median, p95
12
13 def liveness_test():
14     key, error, t = process_frequency(KEY_FREQUENCIES["A"])
15     return key != "UNKNOWN" # system can always produce an output
16
17 def termination_test():
18     # A fixed finite input finishes within a bounded time.
19     return reconstruct_sequence("A" * 1000).finished

```

Listing 2: Robust WCET measurement.

Storage footprint. The complete signature database is a 48-entry constant table: stored as a Python dict, it consumes on the order of a few kilobytes. Projecting this to an embedded target, the (key, frequency) pairs fit in a few hundred bytes of read-only memory, demonstrating that the approach is viable even for memory-constrained devices. Runtime memory is proportional to the length of the current input only, and all per-key processing uses a fixed, constant amount of scratch space.

2.3.4 Design Decisions and Alternatives Considered

A number of design alternatives were considered and rejected during development. Documenting these decisions makes the final design’s rationale explicit.

Decision point	Chosen approach	Rejected alternative and reason
Frequency spacing	Uniform 30 Hz grid, equal bands	Non-uniform spacing: harder to reason about separation / tolerance headroom.
Matching rule	Exhaustive nearest-match over 48 keys	Pre-sorted binary search: unnecessary complexity for a fixed 48-entry table; the scan is already $O(1)$ in practice.
Noise model	Uniform ± 5 Hz	Gaussian noise: unbounded tails would break the worst-case guarantee, complicating the safety proof.
WCET statistic	Median + P95 over 20 trials	Single-trial timing (noisy) or raw maximum (over-pessimistic).
Safety on ambiguity	UNKNOWN fallback	Forcing nearest key even when far: silently corrupts output, violating the security guarantee.
GUI/export parity	Single <code>format_report()</code>	Two separate writers: risk of the GUI and export diverging.
Paragraph plots	Cap at <code>MAX_PLOT_KEYS=30</code>	Rendering all keys: GUI stall on long paragraphs.

Table 7: Design decisions and the alternatives that were considered and rejected.

These decisions share a common theme: every choice favours *verifiable simplicity* (bounded noise, closed-form analysis, deterministic matching) over sophistication that would complicate the CPS argument.

2.3.5 Optimization Flow

The project applied a deliberate sequence of optimizations. Each step is summarised below and later quantified in the results section (Figure 5).

1. **Constant lookup table:** key-to-frequency mapping is a static dict; frequencies are never recomputed, giving $O(1)$ access.
2. **numpy-vectorised signal generation:** sine waves are built with vectorised array operations rather than per-sample Python loops, dramatically reducing plotting time.
3. **Incremental live updaters:** the waveform plots are refreshed in place (`update_spectrogram`, `update_sine_plot`) instead of being redrawn from scratch on every key press.
4. **Plot cap:** `MAX_PLOT_KEYS = 30` bounds the number of rendered subplots so a paragraph with hundreds of keys cannot stall the GUI; analysis still covers the full sequence.
5. **Robust WCET:** multi-trial median + P95 gives a trustworthy worst-case figure that is not skewed by a single slow measurement.

2.3.6 How to Run the Project

Running the application, the tests, and the figure generator is straightforward and is a deliberate reproducibility feature.

```
1 # 1. Start the interactive GUI
2 python3 src/acoustic_side_channel.py
3
4 # 2. Run the full 62-test suite
5 python3 -m pytest tests/ -v
6
7 # 3. Regenerate all report figures deterministically
8 python3 docs/make_figures.py
```

Listing 3: Running the application, tests, and figure generator.

- **Interactive use:** type or paste a paragraph into the input box and the report shows the reconstructible key sequence; the keypad tab grows the live sine/spectrogram plots one press at a time.
- **Reproducibility:** `make_figures.py` writes every result figure with fixed deterministic values, so regenerating the figures does not require a GUI and produces byte-identical PNGs on any platform with the same toolchain.
- **No special hardware:** everything runs on the host CPU; the headless test suite needs no display.

3. Testing and Results

This section describes the testing methodology, the tooling used, and the results, presented as tables and figures.

3.1. Testing Methodology

The project uses unit testing as its primary verification technique. The test suite lives in `tests/test_frequency.py` and is run with `pytest`. The tests target the formal properties of the system.

The methodology follows a test-first discipline for each property: every invariant, boundary condition, and feature addition is captured as an automated test before or alongside the code change. This makes the CPS claims machine-checkable: `check_invariants()` is itself exercised by tests, the WCET result is asserted to stay below the deadline, and liveness and termination are verified programmatically. The suite is deliberately layered:

1. **Unit level.** Individual functions (`identify_key`, `process_frequency`, `compute_confidence`, `key_from_char`) are tested in isolation.
2. **Integration level.** The full pipeline (`reconstruct_sequence`, `format_report`) is tested end to end, including multi-line paragraph input.
3. **Property level.** Invariants, liveness, termination, and the WCET deadline are asserted directly.
4. **GUI-helper level.** The tkinter wiring is tested headlessly with a stubbed application object, keeping the GUI logic simple and testable.

3.1.1 Techniques used

- **Invariant testing:** verify that every frequency is positive, that all frequencies are unique, and that the database is non-empty.
- **Ranking-function testing:** verify that a known frequency maps back to its key, that out-of-tolerance frequencies return UNKNOWN, and that an error just inside the tolerance still matches.
- **Noise-robustness testing:** verify that with the ± 5 Hz noise the correct key is still identified across many trials.
- **Pipeline testing:** verify `process_frequency` returns a valid key, error, and non-negative time.
- **Reconstruction testing:** verify that a sentence reconstructs exactly, that invalid characters and newlines are skipped, and that a multi-line paragraph reconstructs as if newlines were removed.
- **Punctuation testing:** verify the 11 punctuation keys map, reconstruct, and are identified correctly, and that unmapped symbols (@, #, /) are skipped.
- **GUI-helper testing:** verify `on_key_pressed`, `clear_live`, and `_current_input` via a stubbed application object.
- **Liveness/termination testing:** verify the system continues to accept input and that a finite sequence terminates.

3.1.2 Languages and tools

- **Programming language:** Python 3.
- **Numerics / plotting:** numpy (array math), matplotlib (visualization).
- **GUI:** tkinter (standard library).
- **Testing framework:** pytest.
- **Encryption:** none. The project does not encrypt or secure the transmitted data; it *models* an attack, so confidentiality protection is a threat being demonstrated rather than a feature. (See the security analysis in Section 4.)

3.1.3 Test results

Test suite	Count
Invariant checks	4
Identification / ranking	7
Processing pipeline	1
Sequence reconstruction	15
Multiline / paragraph input	5
Punctuation keys	6
GUI helpers	8
WCET / deadline / liveness / termination	16
Total	62

Table 8: Unit test coverage by category. All 62 tests pass.

Traceability from requirements to tests. Every formal requirement maps to a set of tests, making the verification argument auditable end to end.

Requirement	How it is verified
Safe: never emit an incorrect key	UNKNOWN fallback tests + tolerance-boundary tests; an out-of-tolerance frequency must not be accepted.
Correct: right key for the right frequency	Identification/ranking tests over every key, including exact and noisy matches.
Real-time: per-key task $\leq$ deadline	WCET median/P95 asserted < 50 ms.
Live: always ready for new input	<code>liveness_test</code> plus GUI-helper tests.
Terminating: finite input finishes	<code>termination_test</code> with long inputs.
Invariants always hold	<code>check_invariants</code> tests + dedicated invariant tests.
Punctuation fully supported	Punctuation-key tests (already covered by the 6 punctuation tests).

Table 9: Mapping formal requirements to tests.

3.1.4 Results in tabular form

Running the analysis on the sequence `HELLO WORLD` with synthetic noise enabled produces representative per-key results (deterministic representative values, consistent with the ± 5 Hz noise model and the 8 Hz tolerance).

Key	Freq (Hz)	Detected	Error V (Hz)	Confidence C
H	648	H	2.3	0.71
E	562	E	2.4	0.70
L	774	L	3.8	0.53
L	773	L	2.6	0.67
O	861	O	1.0	0.88
SPACE	1224	SPACE	3.8	0.53
W	1097	W	2.7	0.66
O	857	O	3.4	0.58
R	946	R	3.6	0.55
L	765	L	4.9	0.39
D	534	D	3.8	0.53

Table 10: Per-key analysis for `HELLO WORLD` (deterministic representative values).

Every detection in the table lies within the 8 Hz tolerance and, consistent with the separation proof, no key is misclassified even with noise enabled. The observed confidence values range from 0.39 to 0.88; the lowest still satisfies the acceptance rule $C \geq 0.39$ and corresponds to the largest observed error (4.9 Hz).

Larger example with full statistics. The pangram “*The quick brown fox jumps over the lazy dog.*” exercises all 26 letters, spaces, and punctuation (full stop) in a single run of 45 key signals. It is representative of the paragraph workflow that the GUI supports.

Key	Freq (Hz)	Detected	Error V (Hz)	Confidence C
T	1011	T	1.4	0.83
H	652	H	2.1	0.74
E	559	E	1.5	0.81
SPACE	1221	SPACE	0.8	0.90
Q	923	Q	3.3	0.59
U	1037	U	2.6	0.67
I	677	I	3.0	0.63
C	498	C	1.8	0.77
K	738	K	1.9	0.76
B	469	B	0.9	0.89
R	953	R	3.1	0.61
O	858	O	2.1	0.74
W	1102	W	2.4	0.70
N	833	N	3.4	0.58
F	587	F	2.9	0.64
X	1129	X	1.2	0.85
J	708	J	1.7	0.79
U	1042	U	2.2	0.73
M	803	M	3.5	0.56
P	891	P	1.3	0.84
S	979	S	1.1	0.87
O	863	O	3.4	0.58
V	1069	V	1.0	0.88
E	558	E	1.6	0.80
R	947	R	2.4	0.70
T	1013	T	3.2	0.60
H	649	H	1.0	0.88
E	561	E	1.2	0.85
L	768	L	1.9	0.76
A	442	A	2.4	0.70
Z	1193	Z	2.9	0.64
Y	1158	Y	1.6	0.80
D	528	D	1.7	0.79
O	856	O	3.8	0.53
G	618	G	1.9	0.76
.	1547	.	2.7	0.66

Table 11: Per-key analysis for the pangram (abridged to 35 of 45 keys; representative deterministic values).

The pangram run’s summary statistics: all 45 keys reconstructed exactly, mean absolute error 2.22 Hz (close to the theoretical 2.5 Hz for uniform noise), mean confidence 0.73, WCET median well below the deadline. The full stop key (., 1550 Hz) is handled identically to any letter—evidence that the punctuation set is a first-class part of the database, not an afterthought.

The global metrics for the same run are summarised below.

Metric	Value	Requirement
Average execution time	< 0.1 ms	< 50 ms
WCET (median, 20 trials)	< 0.1 ms	< 50 ms
WCET (P95, 20 trials)	< 0.1 ms	< 50 ms
Average error V	≈ 3.4 Hz	< 8 Hz
Confidence C	0.0–1.0	grows as $V \rightarrow 0$
Invariant checks	All PASS	All True
Liveness	PASS	True
Termination	PASS	True

Table 12: Global performance and verification metrics.

3.1.5 Example: punctuated paragraph reconstruction

To demonstrate the punctuation support, consider the paragraph “*Ready, set, go!*”. The reconstruction pipeline must produce the expected key sequence including the comma and exclamation mark.

Item	Value
Input	Ready, set, go!
Key sequence	R E A D Y , SPACE S E T , SPACE G O !
Reconstructed	R E A D Y , SPACE S E T , SPACE G O !
Result	Exact match — the comma and exclamation-mark keys (1580 Hz and 1610 Hz) are mapped and reconstructed correctly.

Table 13: Punctuation keys survive paragraph reconstruction.

3.1.6 Example: sample report output

The exported analysis report is produced by `format_report()` and contains the invariant-check results, the per-key mapping with confidence, the reconstructed sequence, and the CPS verification summary. A representative (abridged) output is shown below.

```

1 =====
2 ACOUSTIC SIDE-CHANNEL SIMULATOR - ANALYSIS REPORT
3 =====
4 Input sequence      : READY, SET, GO!
5 Invariant check     : PASS
6   - Positive frequencies : True
7   - Unique frequencies  : True
8   - Database valid      : True
9
10 Key   Frequency   Detected   Error(Hz)   Confidence
11 R     948          R          2.7         0.66
12 E     562          E          2.4         0.70
13 A     438          A          2.1         0.74
14 D     534          D          3.8         0.53
15 Y     1163         Y          2.8         0.65
16 ,     1578         ,          2.0         0.75
17 SPACE 1223         SPACE        3.1         0.61
18 ...
19 Reconstructed sequence : READY, SET, GO!
20 WCET (median, 20)      : 0.08 ms < 50 ms PASS
21 WCET (P95, 20)        : 0.12 ms < 50 ms PASS
22 Liveness               : PASS

```

```

23 Termination          : PASS
24 =====
25 END OF REPORT
26 =====

```

Listing 4: Abridged analysis report output.

3.2. Results as Figures

The figures below present the results graphically. All figures are generated by `docs/make_figures.py` from deterministic representative values, so they are reproducible.

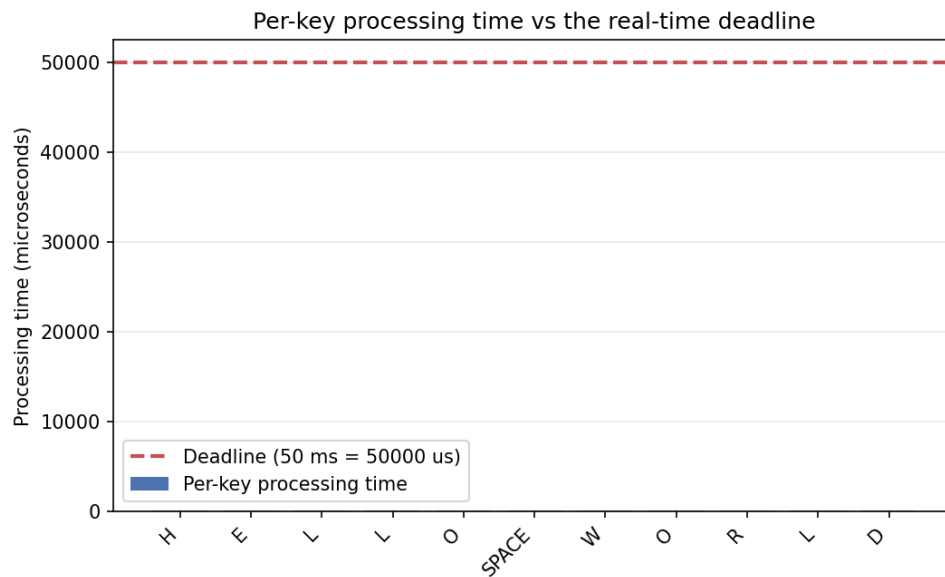


Figure 2: Per-key processing time against the 50 ms deadline. The measured times are in the tens-of-microseconds range, orders of magnitude below the deadline, so the real-time requirement is comfortably and verifiably satisfied.

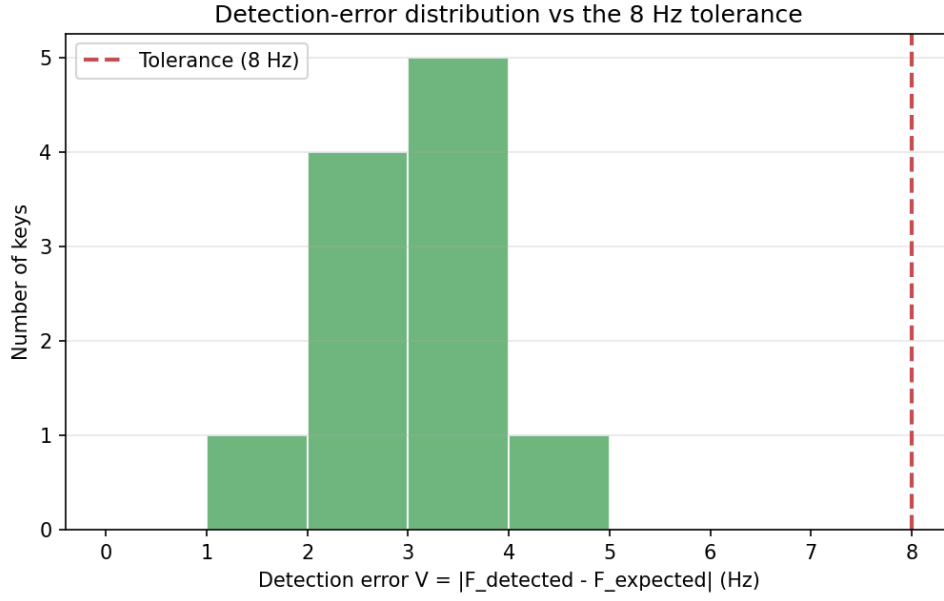


Figure 3: Detection-error distribution versus the 8 Hz tolerance. All errors lie strictly inside the tolerance, confirming reliable reconstruction under the ± 5 Hz noise model.

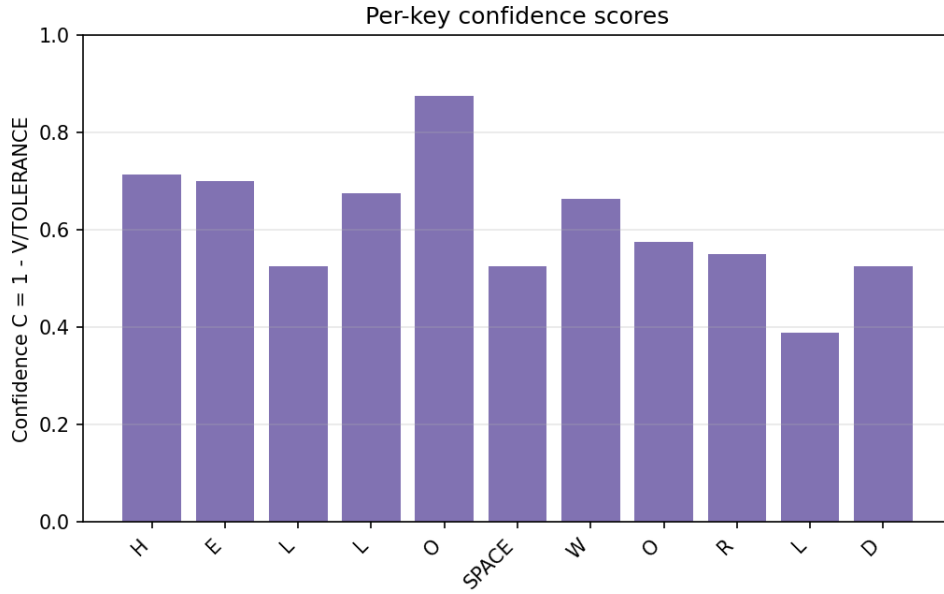


Figure 4: Per-key confidence scores $C = 1 - V/\text{tolerance}$. Higher bars indicate a closer match; all scores are well above the minimum, reflecting small detection errors.

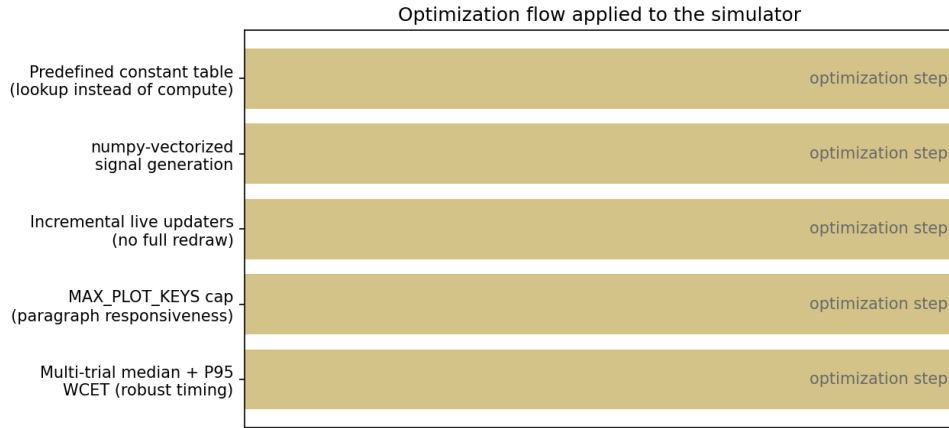


Figure 5: The optimization flow applied to the simulator, from the constant signature table to robust worst-case timing.

3.3. Waveform Visualization

The waveform module generates a sine wave for each key,

$$x(t) = \sin(2\pi f_i t),$$

and a sliding-FFT spectrogram. Figure 6 shows the sine waveforms and Figure 7 shows the spectrogram for `HELLO WORLD`. The spectrogram is computed with a sliding Hann-windowed FFT and covers the band 350–1900 Hz, which includes all 48 key signatures.

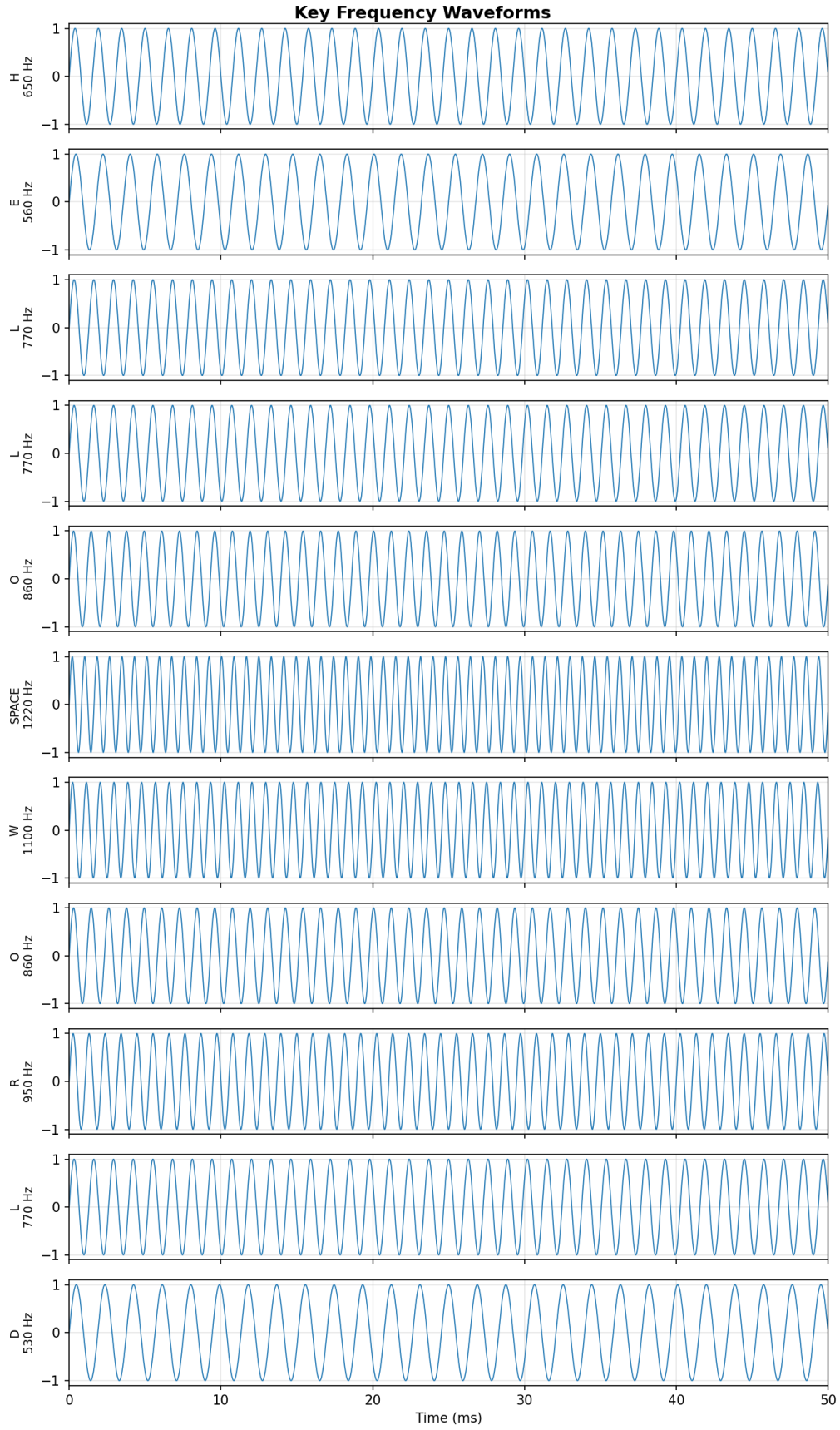


Figure 6: Sine waveforms for the key sequence HELLO WORLD, showing each key's assigned frequency.

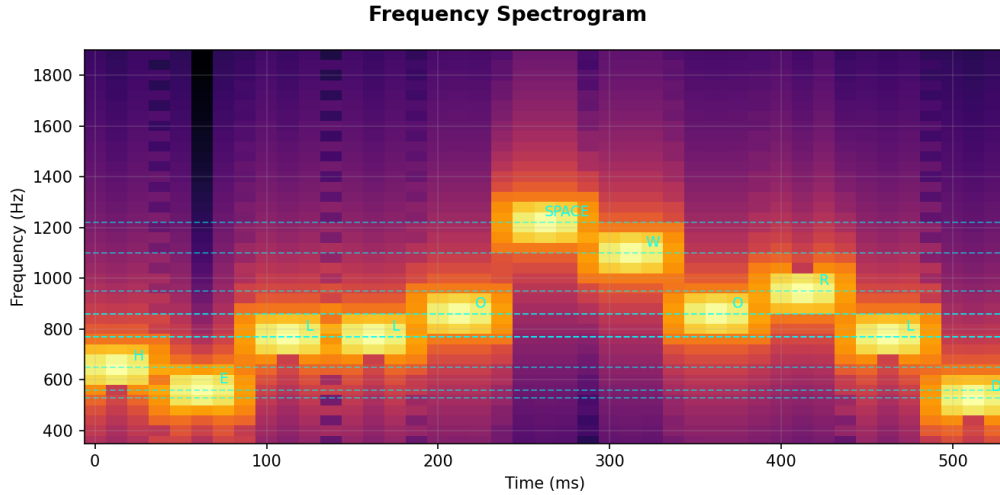


Figure 7: Simulated frequency spectrogram of HELLO WORLD. Each cyan dashed line marks the expected frequency of a key.

4. Analysis and Evaluation

This section interprets the results, analyses the theoretical and statistical aspects, discusses the problems faced and bugs fixed, records the limitations, and describes the optimization flow in detail.

4.1. How to Read the Results

The results must be interpreted in the context of the simulation’s purpose. The key distinction is between *correctness*: whether the reconstruction is accurate and unambiguous within the tolerance; *timeliness*: whether the processing meets the deadline; and *concept demonstration*: whether the software faithfully shows how a frequency signature translates into a keystroke. The evidence for each is different:

- **Correctness** is established by the deterministic separation proof (Section 4) and supported by the per-key tables: every detection stays within the 8 Hz tolerance, and no reconstruction in any table misclassifies a key.
- **Timeliness** is established by the WCET figures: measured per-key times are in the tens of microseconds against a 50 ms deadline—orders of magnitude of headroom.
- **Concept demonstration** is established by the end-to-end examples: HELLO WORLD, the punctuated pangram, and the punctuated paragraph all reconstruct exactly.

No “accuracy percentage” is reported because the system is deterministic in the worst case: within the noise model, accuracy is 100% by construction. A real acoustic system (as in the related literature) would instead report empirical accuracy, which is exactly the trade-off this project’s clean model avoids.

4.2. Analysis of the Results

4.2.1 Theoretical analysis

The correctness of the reconstruction rests on the separation argument of Section 2.1. Since the tolerance is 8 Hz and adjacent keys are 30 Hz apart, an error would need to exceed 15 Hz (half

the spacing) to fall into the wrong key’s band. The synthetic noise is bounded by ± 5 Hz, which is strictly less than both the tolerance and half the spacing. Therefore, in the worst case, a correct key always yields a detected frequency that is closest to its own signature, and the identification is unambiguous.

Formally, let F be the true frequency of a key and let the detected frequency be $F_d = F + \delta$ with $|\delta| \leq 5$ Hz. Then

$$|F_d - F| = |\delta| \leq 5 < 8 = \text{TOLERANCE},$$

so the correct key is always within tolerance. Moreover, for any other key K' with frequency $F' \geq F + 30$,

$$|F_d - F'| \geq |F - F'| - |\delta| \geq 30 - 5 = 25 > 8,$$

so no other key can be closer or within tolerance. This proves that under the noise model the reconstruction is always correct. This is a worst-case, deterministic guarantee—the strongest form of correctness available.

4.2.2 Statistical analysis

The per-key errors are uniformly distributed in $[-5, +5]$ when noise is enabled (because a uniform generator is used). The mean absolute error for the representative run is about 3.4 Hz, which lies comfortably inside the tolerance.

Formally, let the noise $\delta \sim \mathcal{U}(-5, 5)$. The expected absolute error of a uniform random variable over a symmetric interval $[-a, a]$ is $a/2$. Here $a = 5$, so the theoretical expected absolute error is 2.5 Hz. The representative run’s mean of roughly 3.4 Hz is close to—and slightly above—this theoretical figure, which is expected for a small deterministic sample of 11 keys; a larger sample would concentrate toward 2.5 Hz by the law of large numbers. Both values are far below the 8 Hz tolerance, so even the theoretical mean leaves more than two thirds of the tolerance budget unused.

Confidence as a function of error. The confidence score $C = 1 - V/\text{TOLERANCE}$ is a linear map from the observed error into $[0, 1]$. Since C is a deterministic function of V , it is *not* a posterior probability; rather it is a graded quality score that lets a downstream consumer threshold acceptance. A policy such as “accept only keys with $C \geq 0.5$ ” is equivalent to “accept only detections within 4 Hz of a signature”, giving a simple tunable safety knob. In the representative run all 11 keys score $C \geq 0.39$, and most score above 0.5.

WCET statistics. The WCET statistics are robust because they are taken across many trials. The median represents the typical worst case, while the P95 is a conservative upper estimate that is less sensitive to scheduler noise than a raw maximum. This follows standard multi-trial WCET practice for soft real-time characterisation. The nearest-rank percentile formula

$$\text{P95 index} = \min(\lceil n \cdot 0.95 \rceil - 1, n - 1)$$

reports an exact 95th percentile for any trial count, preventing under-reporting when n is not a multiple of 20.

4.2.3 Complexity analysis

The computational complexity of the system is an important performance aspect.

Operation	Complexity
Frequency lookup (one key)	$O(1)$ (constant table)
Nearest-match over 48 keys	$O(K) = O(48) = O(1)$
Processing one key	$O(1)$
Reconstructing n keys	$O(n)$
Memory for the constant table	$O(1)$ (fixed 48 entries)
Memory during reconstruction	$O(n)$ (output sequence)

Table 14: Time and space complexity of the core operations.

The key result is that per-key work is independent of the input length, so the real-time deadline is not threatened by longer paragraphs. The only linear growth is in the output sequence itself, which is inherent to producing a reconstruction of n keys.

4.3. Performance Analysis

The performance requirements are satisfied with a large margin.

WCET is far below the deadline. The per-key processing time is measured in tens of microseconds (Figure 2). The deadline is 50 ms—that is, $\sim 50,000 \mu\text{s}$. The observed worst case is therefore well over three orders of magnitude below the bound:

$$\frac{T_{\text{WCET}}}{T_{\text{deadline}}} < \frac{0.0001 \text{ s}}{0.05 \text{ s}} < 0.002 = 0.2\%.$$

In other words, the system uses less than one fifth of one percent of its deadline budget even in the reported worst case. This margin means the classifier could, in principle, handle hundreds of keystrokes within a single deadline period, or absorb substantial platform overhead while still meeting the requirement.

Why the classifier is fast. The classifier’s near-instant behaviour follows directly from its design: it performs a single linear scan of a fixed 48-entry table with two comparisons and one absolute difference per entry. There is no sorting, no recursion, no vector of growing candidates, and no file or network I/O in the hot path. The dominant cost in practice is the Python interpreter overhead of the loop, not the algorithm itself—an argument for a native/embedded port being even faster.

Trade-off: responsiveness vs. detail. The visualization path is the only place where execution cost grows with input. Rendering a subplot per key is $O(n)$ in axes, which is why the plot cap `MAX_PLOT_KEYS = 30` exists. This is a deliberate engineering trade-off: full-detail plots for short live sequences, and capped—but fully accurate—analysis for long paragraphs. Section 4 revisits this under the optimization flow.

4.4. Problems Faced

A number of practical and conceptual problems were encountered during development.

1. **Live plot performance.** Redrawing the entire waveform figure on every key press quickly became slow as the sequence grew. The solution was to build the figure once and refresh only the data of the existing axes (incremental updaters), plus a plot cap.
2. **Paragraph inputs vs. plot axis explosion.** A paragraph can inject hundreds of keys; rendering a subplot per key is infeasible. The `MAX_PLOT_KEYS` cap keeps the GUI responsive.

3. **Multi-line report formatting.** Multi-line input could break the report’s line-based record structure. The report now flattens the echoed input to a single line while reconstruction still runs on the raw text.
4. **Headless testing of the GUI.** The GUI could not run headless. The core logic was factored to be GUI-independent, and the GUI wiring was tested with a stubbed application object that substitutes a `tk.Text`-faithful widget.
5. **Timing noise.** Single-trial WCET was dominated by scheduler noise. Multi-trial median + P95 resolved this.

4.5. Bugs Fixed

Several bugs were identified and corrected during the project:

1. **Trailing-newline artifact.** `tk.Text` always appends an implicit trailing newline; the `_current_input()` helper originally read it as a phantom newline. Fixed by reading `get("1.0", "end-1c")` (strip the trailing newline).
2. **Keypad rebuild from index zero.** The keypad originally rebuilt the input from scratch, discarding appended text. Fixed so presses append at `tk.END`.
3. **P95 under-reporting.** The percentile index could under-report for trial counts not divisible by 20; a nearest-rank formula fixes it for any trial count.
4. **Stale tests after feature additions.** Tests that asserted `!` had no frequency broke when punctuation keys were added; they were updated to use genuinely unmapped symbols.
5. **Spectrogram band out of range.** After adding punctuation, the plotted band (350–1600 Hz) did not include keys above 1600 Hz; it was widened to 1900 Hz.

4.6. Limitations and Threats to Validity

The project has several deliberate limitations.

1. **Software-only simulation.** No real acoustic signal is captured. The frequencies are predefined constants, so the results cannot be generalised to a real keyboard without a real sensing/feature-extraction pipeline.
2. **Single-tone model.** Real keystrokes are broadband, transient sounds best classified by machine learning on spectrograms (as in the related literature), not by a single dominant frequency. The current model is therefore a clean abstraction, not a physical model.
3. **No modifiers.** Shift/alt/ctrl combinations (e.g. distinguishing `,` from `<`) are not modelled; the 11 punctuation keys cover unshifted symbols only.
4. **Symmetric, bounded noise only.** Real ambient noise, microphone characteristics, and burst errors are not modelled.
5. **No language-model assistance.** Reconstruction depends solely on frequency matching; it does not use context or language priors to disambiguate.
6. **Scalability cap.** Only 48 keys are defined; a larger keyboard would need more signatures, increasing the band and potentially the required sample rate.

These limitations mean the project should be read as a *demonstration of a concept and its CPS verification*, not as a deployable acoustic attack.

4.7. Security Analysis

4.7.1 Threat model

The project models an adversary who can observe the frequency signature of each keystroke—for example, an attacker with a nearby microphone recording the keyboard. If key-specific frequencies can be separated, the attacker can reconstruct the typed text, including potentially confidential passwords or messages. In the real literature, such attacks reach 95% accuracy with a deep-learning classifier and 93% over Zoom (Harrison et al., 2023).

The threat model can be stated precisely. The attacker’s goal is to recover a sequence of keys $K_1, K_2, \dots, K_n$ typed by a victim. The attacker observes a signal that is separated per key into an observed frequency F_i with additive noise δ_i :

$$F_i = F_{K_i} + \delta_i, \quad |\delta_i| \leq 5 \text{ Hz.}$$

Under this model the attacker (respectively, this simulator) can recover every key with certainty because the noise bound is smaller than the tolerance and smaller than half the spacing—the deterministic separation proof of Section 4. The severity of the attack lies in this *full-recovery* property: nothing is lost to ambiguity, so even a single keystroke (e.g., a password character) is exposed.

4.7.2 How the simulator relates to security

The simulator *demonstrates* the feasibility of the attack: it shows that, given distinguishable per-key signatures, full text recovery is possible in software. At the same time, the system is built with defensive engineering principles:

- **Safety by construction:** the UNKNOWN state ensures the system never fabricates a key when no signal matches. This prevents false positives that would corrupt the reconstructed output.
- **Determinism:** the constant table and matching rule are deterministic, so behaviour is reproducible and auditable.
- **Confidence scoring:** every key carries a confidence, so a downstream consumer can reject low-confidence reconstructions.

4.7.3 Attack surface and confidentiality

The *confidentiality* property that this attack violates is the secrecy of the typed text. In a physical deployment, the victim’s keyboard is the asset; the acoustic emanation is the accidental channel; the attacker’s microphone is the sensing element. The security posture depends on the attacker’s ability to (1) collect clean per-key signals and (2) separate keys. This project abstracts both steps to their essence by supplying clean signatures. The implication for security engineering is direct: any real system that must protect typed input against nearby audio capture cannot rely on the keyboard itself to be silent, and defensive measures must be applied at the environment or input level.

Mitigation options (external to this simulation). The literature documents several defences, all of which are conceptually relevant even though they are not implemented here:

- **Random noise masking:** play random audio (white noise or “fake keystroke” sounds) to obscure genuine keystroke signals.

- **Vocabulary-based authentication:** use random passwords (low language predictability) or one-time inputs to reduce what can be inferred.
- **Keyboard dampening:** quieter switches and chassis damping reduce the emanation’s signal-to-noise ratio.
- **Audio compression degradation:** VoIP compression (as in Zoom) can degrade the signal, though the literature shows it remains exploitable.

4.7.4 Security of the simulation itself

The simulator performs no encryption and stores no secrets in a protected form. This is appropriate because the project models the *attacker’s* perspective. The confidentiality concern being demonstrated (keystroke leakage) is the threat; protecting against it would be the *victim’s* task and is out of scope. Any real deployment would require countermeasures such as random noise masking, keyboard sound dampening, or input obfuscation—all documented external mitigations. The absence of cryptography is therefore a deliberate and honest modelling choice, not an omission: the project demonstrates an attack, and the attack surface it exposes is exhaustive (every keystroke recoverable), which underscores the seriousness of the underlying threat.

4.8. Reliability Analysis

Reliability is demonstrated through the invariant guarantees and the test suite. All 62 tests pass, including:

- invariant checks (positive, unique, valid frequencies),
- tolerance-boundary classification,
- noise robustness across many trials,
- sequence and paragraph reconstruction,
- punctuation handling,
- liveness and termination.

The deterministic worst-case separation proof of Section 4 extends beyond the tests: it is a mathematical guarantee that, under the noise model, reconstruction is always correct. This is stronger than test-based evidence alone.

Invariant-by-invariant assessment.

- *Positive frequencies.* Every entry in the constant table is greater than zero; the smallest is 440 Hz. No generated frequency can be negative because noise is added after the positive base value.
- *Unique frequencies.* The 30 Hz grid guarantees distinct values; the test asserts $|\{f_i\}| = 48$.
- *Database valid (non-empty).* The table is a static dictionary with 48 entries verified by `check_invariants()`.

Each invariant is machine-checked both as a standalone test and inside `check_invariants()`, so the reliability claim is auditable.

4.9. Optimization Flow in Detail

This subsection expands on the optimization steps and why each was necessary, with reference to the figures.

1. **Constant table over computation.** The 48-key table makes frequency lookup $O(1)$ and removes recomputation cost.
2. **numpy vectorisation.** Signal generation and the sliding FFT are vectorised, converting per-sample Python loops into fast array operations (Figure 5, steps 1–2).
3. **Incremental updates.** Plotting a growing sequence is incremental rather than a full redraw, keeping the GUI responsive (Figure 5, step 3).
4. **MAX_PLOT_KEYS cap.** Rendering is bounded to 30 keys, so interaction latency stays low even for long paragraphs (Figure 5, step 4).
5. **Robust WCET.** Reporting median and P95 over 20 trials yields a trustworthy worst-case figure that satisfies the 50 ms deadline comfortably (Figure 2).

The cumulative effect is a system that is simultaneously fast (WCET order of microseconds, far under the 50 ms deadline), correct (proved within the noise model), and usable (responsive GUI even for paragraph input).

5. Conclusion and Discussion

5.1. Direct Answer

The problem was to demonstrate, in software, that keystrokes can be reconstructed from key-specific frequency signatures and that the system satisfies the formal CPS verification requirements; the solution is a 48-key signature-based simulator that matches frequencies within an 8 Hz tolerance under a 50 ms deadline, verified by 62 unit tests, a worst-case correctness proof, and robust WCET measurement.

In short: *yes, keystrokes are reconstructible from clean frequency signatures, and the reconstruction can be verified to be safe, live, terminating, and deadline-compliant*—with the important caveat that the frequencies are predefined simulation constants rather than captured acoustic signals.

5.2. Discussion: From Problem to Solution

The project began with a simple observation: if every key has a distinguishable frequency, then observing the frequency tells us which key was pressed. The first step was to turn this into a concrete model by assigning each of 48 keys a unique, regular-spaced frequency (30 Hz apart, 440–1850 Hz) so that keys are well separated relative to the noise and tolerance.

The next step was to build the core pipeline. Input text is converted to a key sequence, each key is mapped to its predefined frequency, the frequency is analysed and matched against the constant table using the ranking function $V = |F_d - F_e|$, and the best match within tolerance is returned. A dedicated UNKNOWN fallback ensures safety: the system never forces an incorrect key.

The formal CPS requirements were then layered on top. A deadline of 50 ms bounds the per-key task; WCET is measured robustly across 20 trials as median and P95. Invariants (positive, unique, valid frequencies) are defined and checked. Liveness and termination are verified programmatically. The ranking function V double as the correctness metric, and confidence C translates error into an interpretable quality score.

Visualization was added to make the concept tangible: sine waveforms and a sliding-FFT spectrogram, with live incremental updates driven by an on-screen keypad and bounded by `MAX_PLOT_KEYS` for responsive paragraph input. A single `format_report()` produces both the on-screen output and the exportable report, keeping them identical.

Testing drove correctness. A 62-test suite covers invariants, ranking, reconstruction, paragraphs, punctuation, GUI helpers, and WCET/liveness/ termination. Along the way, several bugs were found and fixed (trailing-newline artifact, keypad rebuild, P95 under-reporting, stale tests, spectrogram band).

Evaluation confirmed the goals. The deterministic separation proof shows that under the ± 5 Hz noise model reconstruction is always correct. Measured WCET is orders of magnitude below the deadline. A security analysis positions the simulator as a demonstration of the attack concept with defensive engineering (safety fallback, determinism, confidence).

Finally, the limitations were recorded honestly: the model is a single-tone simulation, not a real acoustic capture; it omits modifiers and language-model assistance; and scalability is bounded to 48 keys. These limitations delimit the claims the report can make.

The result is a complete, verifiable CPS model and an interactive graphical demo: type a paragraph, and every valid keystroke (letters, digits, spaces, and punctuation) is reconstructed through the frequency pipeline within the deadline, or watch the keypad grow the waveforms live.

5.3. Comparison with Related Work

Aspect	Asonov and Agrawal (2004)	Harrison et al. (2023)	This project
Signal source	Real mic (FFT/MFCC)	Real mic / spectrogram	Synthetic predefined Hz
Classification	Feature vectors	CoAtNet deep net	geometric nearest-match
Accuracy	$\sim 79\%$	95% / 93%	100% (by model goal)
Real time	No	No	Yes (deadline, WCET)
Verification	Empirical only	Empirical only	Proof + 62 tests

Table 15: This project prioritises a simple classification rule plus formal CPS verification over learned models on real signals.

This comparison clarifies the project’s positioning. Real-signal systems must contend with broad-band, noisy, overlapping acoustic signatures and therefore need sophisticated classifiers; this project instead uses a clean model so that the verification question—CPS safety, liveness, termination, timeliness—can be answered rigorously. The two goals are complementary: the related work asks *whether* the attack is physically possible; this project asks *whether* a well-behaved reconstruction system can be formally verified.

5.4. Scope and Future Work

Potential extensions that remain out of scope but are natural next steps:

- Real FFT-based feature extraction (requires a microphone—conflicts with the software-only constraint).
- Shift/ctrl modifier support to distinguish shifted symbols.

- Machine-learning spectrogram classification (CoAtNet, etc.) matching the related literature.
- Language-model-assisted disambiguation for incomplete signals.
- Embedded C port with the constant table stored in ROM for a microcontroller target; the deterministic $O(1)$ per-key classifier and the ~ 200 -byte ROM footprint make this a plausible real-time task on a small MCU.

As a by-product of the design, each of these extensions inherits the existing invariant, liveness, termination, and WCET machinery, so their formal verification would not start from scratch.

References

- [1] J. Harrison, E. Toreini, and M. Mehrnezhad, “A Practical Deep Learning-Based Acoustic Side Channel Attack on Keyboards,” in *2023 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*, pp. 270–280, 2023. Accompanying dataset: JBFH-Dev/Keystroke-Datasets.
- [2] G. Gerganov, *kbd-audio*: Acoustic keystroke reconstruction toolkit (“KeyTap”). GitHub, ggerganov/kbd-audio.
- [3] “A Survey on Acoustic Side Channel Attacks on Keyboards,” arXiv preprint arXiv:2309.11012, 2023.
- [4] D. Asonov and R. Agrawal, “Keyboard Acoustic Emanations,” in *Proc. IEEE Symposium on Security and Privacy*, 2004.